usually implemented as immutable objects, so extracting a substring would require linear time and extra space, `O(m)`, for each call. Luckily the solutions is simple: we can pass to the recursive call a reference to the beginning of the string and the index of the next character. This way, even this operation can be considered `O(1)`.

Since each call can be implemented in such a way to require amortized constant time, the whole search takes `O(m)` amortized running time.

6.2.3 Insert

Like `search`, `insert` can better be defined recursively; we can distinguish two different scenarios for this method:

- The trie already has a path corresponding to the key to be inserted: if that's the case, we only have to change the last node in the path, making it a *key node* (or, if we are indexing a text, adding a new entry to the list of indices for the word).
- The trie only has a path for a substring of the key: in this case, we will have to add new nodes to the trie.

Figure 6.8 illustrates an example of the former situation, where a call to insert("anthem") on the trie in figure 6.7 will result in adding a new branch to one of the leaves of the tree.

The method, as it's also possible to see from listing 6.5, mainly consists of two steps: in the first step, we search for